

Engaging Cloud-Based LLMs For Zero-Day Threat Detection In IoT Devices

Carston Dastrup

6/22/26

IT 6740

Advanced Network Defense and Countermeasures

Table of Contents

Executive Summary

1. Introduction

1.1 Background and Problem Statement

1.2 Project Objectives and Scope

2. Methodology & System Architecture

2.1 Dataset Selection and Stream Processing

2.3 Data Optimization Strategy and Budget Constraints

3. Results and Empirical Analysis

3.1 Global Incident Severity Distribution

3.2 Anomaly Mapping to MITRE ATT&CK

3.3 Micro-Level Telemetry Verification

4. Discussion & Future Work

4.1 Operational Efficiency and Budget Trade-offs

4.2 Architectural Vulnerabilities: LLM Hallucinations

5. Conclusion

References

Executive Summary

Smart devices, also known as IoT (Internet-of-things) devices, are continuing to grow at a rampant pace. As more and more devices connect to networks, it becomes paramount that cybersecurity experts remain ever vigilant. In Verizon's 2025 Data Breach Investigations Report (DBIR), they reported an eightfold increase in the exploitation of edge devices, routers, and IoT network perimeters compared to the previous year (Verizon, 2025). This increase showcases there is an ever expanding attack surface within IoT devices which threat actors are utilizing. Defensive measures against IoT attacks are largely reactive. In this paper, we will explore utilizing Google Gemini to both increase the speed at which cybersecurity analysts detect and respond, and even proactively prevent the attacks from taking place. We will further centralize the data into Splunk to produce actionable security dashboards.

1 - Introduction

1.1 Background and Problem Statement

In 2025 Verizon released their most recent Data Breach Investigations Report (DBIR). In this report it was noted there had been an eightfold increase in the exploitation of edge devices, routers, and IoT network perimeters compared to the previous year (Verizon, 2025). This is a significant concern, as it adds further workload on an already limited bandwidth, while showing there is an expanding and concerning attack-surface within IoT devices which requires additional operational capacity. A major reason for this expansion is that IoT devices are notoriously difficult to effectively monitor. Some IoT devices logs are not able to be monitored by end-users. Monitoring these devices requires much more focus on the network packets passing to and from the device. This blindspot further complicates the monitoring of these devices. Another major concern is many IoT devices may go completely unpatched by the manufacturer, leaving IT and SECOPS teams with technology bloat that can also be utilized as an attack vector.

According to research highlighted by Yang (2017), roughly 15% of device owners do not modify the factory-default login credentials on their smart devices. This failure leads to a massive attack-surface. In addition to users neglecting to update the factory-default credentials, many IoT devices come shipped with largely generic and overused credentials. This was further outlined by Yang, “the five sets of default username and passwords, according to the report, are also support/support, admin/admin, admin/0000, user/user and root/12345” (2017, para. 3). This overuse of generic logins allows for incredible low-effort dictionary attacks to quickly breach IoT devices. Even if login attempts have an escalating wait time, or a lock-out period after a few failed login attempts, with a short list of 5 combinations, lock-outs will have minimal impact on

threat actors, as after they are locked out, it is highly likely they have already exhausted the 5 most common defaults and the threat actor can move onto the next target.

1.2 Project Objectives and Scope

In a blog post by Searchlight Cyber (an industry recognized cybersecurity firm), it was posited that attack surfaces have grown, and are continuing to grow faster than the capacity of SOCs. This has forced many security teams to operate under an entirely reactive defensive posture (Searchlight Cyber, n.d.). Understanding this gap, and the increased growth and sophistication of modern LLMs, this project investigates if it is possible to utilize a cloud-based LLM to quickly triage and detect anomalous behavior within IoT devices. Google Gemini was chosen as the LLM for this project. Furthermore, this pipeline is engineered for its output to enforce a clear and deterministic output schema. This schema is structured to be readily ingested into Splunk to create quick, effective, and actionable security dashboards. Several datasets were examined, but it was determined that the project would use the open-source IoT-23 network telemetry logs as

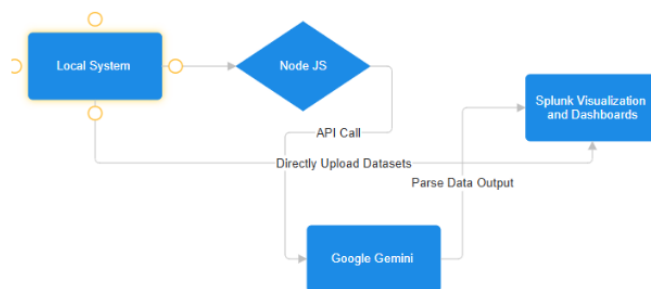


Fig 1. Project Technical Workflow

the input for Google Gemini (Garcia et al., 2020). [Node.js](#) was chosen to implement the pipeline architecture. Figure 1 illustrates the technical workflow of the pipeline.

2 - Methodology & System Architecture

2.1 Dataset Selection and Stream Processing

To ensure a rigorous and realistic testing ground for the LLM pipeline, the underlying architecture relies on the well-known, open-source IoT-23 dataset (Garcia et al., 2020). The IoT-23 dataset was published by the Stratosphere Laboratory at Czech Technical University, and consists of network captures. These captures contain 20 distinct malicious packets spanning from IoT botnets, such as Mirai, Torii, and Gafgyt, in addition to three distinct benign baseline profiles, which were captured from real consumer hardware endpoints.

It is critical to note that this project intentionally does not utilize the entirety of the dataset, as this encompasses over 30 gigabytes of raw PCAPs and text logs. These logs comprise hundreds of millions of rows of connections and packets, and attempting this through Google Gemini would lead to immense token consumption, large financial costs, and severe API timeout thresholds. These drawbacks are entirely unfeasible for production enterprise environments or academic research frameworks without a direct agreement with the cloud-based LLM providers. This project is more to show a proof-of-concept in whether it is possible to utilize cloud-based LLMs for IoT triage and anomaly detection.

Furthermore, rather than feeding raw packet binaries directly to the LLM, the ingestion method targets isolated, highly structured connection telemetry logs which were generated by the Zeek network analysis framework. The architecture targets two major categories of telemetry fields from the logs to construct the analytical context of the LLM.

- Temporal and Layer-4 Primitives
 - Source and destination IP addresses
 - Transport protocols
 - Destination ports
 - Timestamps
- State and Volume Metrics
 - Packet frequencies
 - Total sent/received byte counters
 - Connection state flags.

In order to operationalize the ingestion framework across the different malicious and

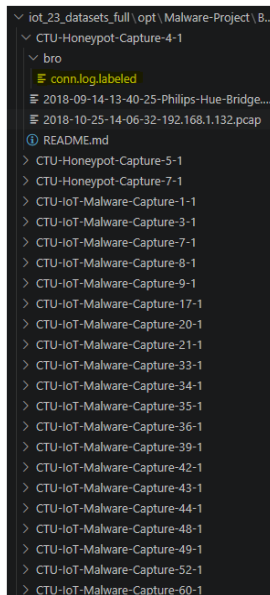


Fig 2. File Structure

benign subdirectories of the IoT-23 repository, the engine needed to navigate a multi-layered, nested storage directory structure. Rather than manually targeting the isolated log files, a custom automated script was used to dynamically traverse through the file structure to find the conn.logs. These logs were consistently in a subfolder labeled, “bro”, enabling an efficient traversal executed by the engine to find the logs. As illustrated in Figure 2, the pipeline must programmatically crawl through individual scenario folders in order to parse the underlying conn.log streams.

Filtering these fields was done to directly align with the operational security principles which are outlined in Internet Engineering Task Force (IETF) RFC 8520, which defines Manufacturer Usage Description (MUD) specifications (Lear et al., 2019). The MUD

framework establishes that an IoT device's intended network behavior can be deterministically profiled and restricted based on its expected Layer-4 protocols, destination service ports, and authorized target IP addresses (Lear et al., 2019). When these connection primitives are isolated from the IoT-23 Zeek logs, the pipeline creates a low-noise and high-density footprint, which allows cloud-based LLMs to cross-reference the traffic against a set of standard baseline expectations. This process eliminates the need for the LLM to further ingest bloated packet payloads, improving performance and reducing cost.

A hurdle which required addressing pertains to Node itself and the Zeek conn.log captures. Due to the log captures within the IoT-23 repository scaling to enormous sizes, it is not possible to load the entire log system into RAM using traditional synchronous methods, such as the [Node.js](#) native `fs.readFile()`. Using this method would quickly trigger a fatal out-of-memory heap allocation exception, crashing the engine before the LLM could finish writing the output. To mitigate this, the orchestration layer is engineered to utilize an asynchronous, event-driven I/O model. The ingest script instead implements the native `fs.createReadStream()` module coupled with the readline interface. This instead pulls data from the files as chunks, and translates them into text lines on the fly. When instead processing the telemetry data on a line-by-line basis we fully eliminate the possibility of triggering an out-of-memory heap allocation exception.

2.2 Data Optimization Strategy and Budget Constraints

A major caveat to utilizing cloud-based LLMs is ultimately cost. Cloud-based LLMs can offer impressive contextual reasoning capabilities as it pertains to security telemetry

triage, but transitioning from a proof-of-concept to production deployments requires a strict fiscal discipline. Unlike tools like Suricata, which is a local, signature-based intrusion detection system (IDS), cloud-based LLMs operate under a variable-cost usage model, which is directly tied to runtime token volume consumption. This can lead to costs being highly ambiguous without strict limitations on token usage.

Recent analyses of infrastructure from Splunk highlight that consumption-based token models introduce severe, non-linear budget overruns when applied to high-volume operational workflows which lack explicit constraint layers (Lacey, 2026). Security logging systems generate immense continuous streams of text data, and unchecked automation pipelines can cause expenditures to balloon to unexpected amounts before any cloud alerts could trigger. This has recently been reported on, when a currently unnamed enterprise client racked up an astonishing \$500 million bill against Claude AI in a single month because they failed to set usage or spending limits for its employees (Yahoo Finance, 2026). This demonstrates the necessity to add usage and spending caps if utilizing cloud-based LLMs for security triage.

Given this is a proof-of-concept with a limited budget, there was a need to enforce strict cost management. It was decided to utilize Google Gemini's Pro LLM, the reason for this is due to better reasoning, which should lead to improved triage. The drawback is the cost. According to the official Google API pricing data, the premium reasoning model (Gemini Pro) costs \$1.25 per million input tokens and \$5.00 per million output tokens, meanwhile the efficiency-focused model (Gemini Flash) costs far less at \$.075 per

million input tokens and \$.30 per million output tokens. Effectively, utilizing Gemini Pro costs 16.6 times more to use than Flash. Due to the cost difference, it was decided to limit the proof-of-concept to a budget of 1,000 lines. In fact, creating a line budget is effectively mandatory if an enterprise-level organization wishes to utilize Gemini Pro as compared to Gemini Flash.

In fact, limiting the budget to a 1,000-line slice mitigates large financial spends, while still ensuring an effective monitoring posture. This is due to the specific behavioral characteristics of the threat profiles within many IoT devices. Malicious IoT activity tends to lean towards highly repetitive and programmatic attacks such as denial-of-service (DDoS) attacks, automated botnets, and horizontal port scanners. When a resource-constrained consumer IoT device is compromised by malware strains like Mirai, the malware will execute loop-based instructions which will generate thousands of identical connection rows within the Zeek text streams. An example is a continuous SYN connection attempt against target ports like Telnet/23 or HTTP/80 at high speeds.

3 - Results and Empirical Analysis

3.1 Global Incident Severity Distributions

The primary objective was to feed the 1,000-line Zeek telemetry logs into the orchestration engine. Instead of a dependence on basic binary alert matching, the framework classified the incident state by a spectrum of severity (Benign, Low, Medium, High, Critical). To ensure statistical accuracy, this approach mirrors the data structure paradigms which were validated by the CICIoT2023 dataset, which is viewed as an industry standard for the profiling of large-scale attack metrics over numerous heterogeneous IoT topologies (Neto et al., 2023)

When executed against the 1,000 lines of IoT-23 Zeek logs, the Gemini Pro model was successfully able to handle the high-volume network event histories. Across the proof-of-concept, malicious botnet captures, such as Gafgyt or Mirai, were triggered as High or Critical alerts. The LLM was able to see the pattern of attack through the dense connection frequencies over brief time frames.

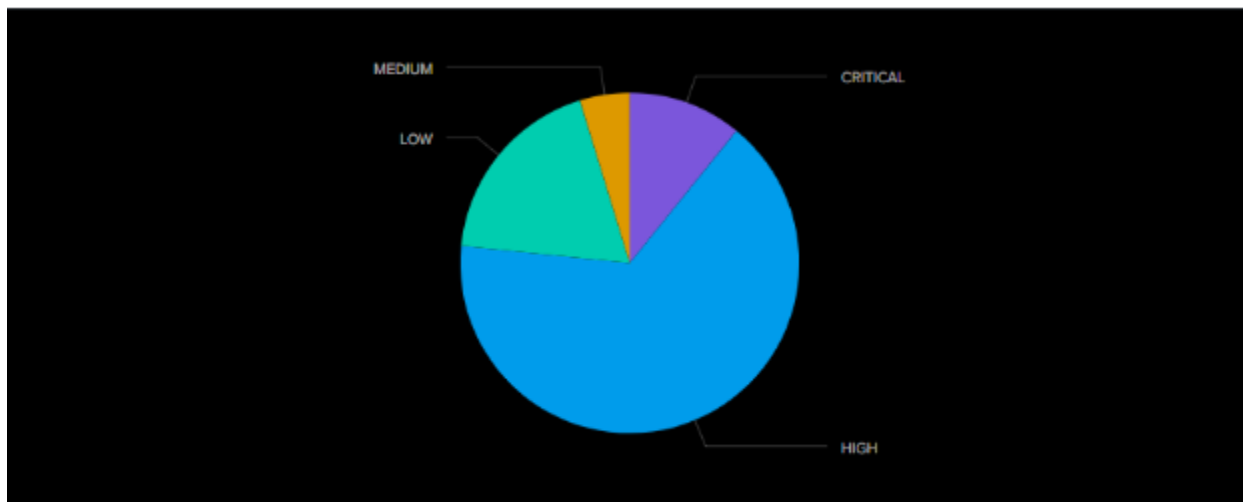


Fig. 3 - Gemini IoT-23 Severity Aggregation

As shown above in Figure 3, the classifications were ingested into Splunk, where a readily clear and actionable breakdown of severity alerts from the provided dataset was

created. The benefits here are clear for an SOC team, they do not need to put in the operational effort to handle the triage of heterogeneous IoT devices.

3.2 Tactical Anomaly Mapping to MITRE ATT&CK

After severity determination is completed, the LLM will then further contextualize the connection telemetry by translating directly into the standard MITRE ATT&CK for Enterprise framework. (The MITRE Corporation, 2025). By putting triage reliance onto the LLM, an SOC team could effectively create an active and autonomous agent, which could be readily utilized in any network defense. This agent would be able to work seamlessly in the background to autonomously triage logs. When analyzing the 1,000-line Zeek logs, the LLM, in this case Gemini Pro, was able to map the following observed anomalies to their associated MITRE ATT&CK techniques. Figure 4 shows the aggregation of the techniques flagged by the LLM.

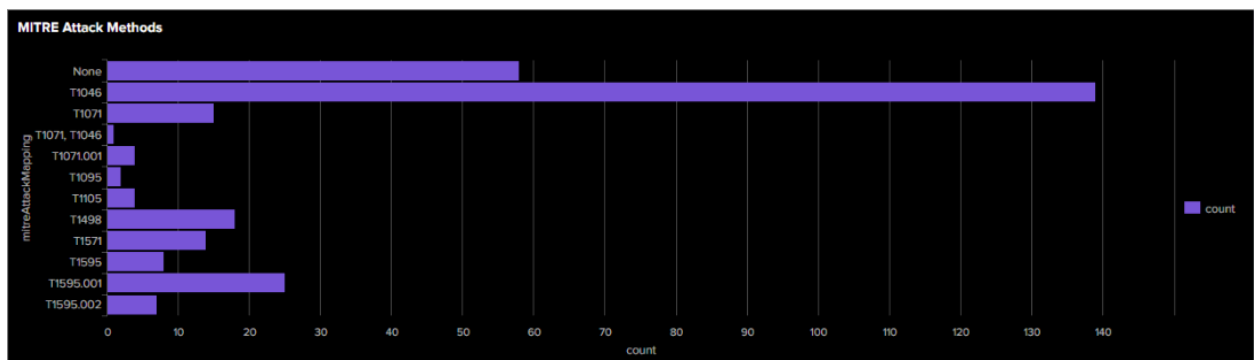


Fig. 4 MITRE Aggregation

Again, the LLM was effectively able to quickly triage the Zeek logs, and when ingested into Splunk, create readily-actionable aggregation charts. These charts could easily be translated into alerts, which could alert engineering personnel. This would allow the SOC to be able to more readily react to anomalous IoT behavior.

3.3 Micro-Level Telemetry Verification

While high-level, aggregated actionable summaries are critical to an SOC team, the LLM was also able to provide additional context into its decision making, in addition to a confidence score. Let’s look at a specific example here, which describes a series of connections being made by the IoT device at address 192.168.1.195. The LLM was able to determine the IoT device was persistently communicating with a single external ip address (185.244.25.245) over the TCP port 6667. Figure 5 shows an actionable tabular view displaying timestamp, technique mapping, confidence score, and the summary analysis of the LLM.

timestamp : 1	confidenceScore : 1	mitreAttackMapping : 1	summary/analysis : 1
2026-07-06T17:01:16.069Z none	100	T1071	The IoT device at 192.168.1.195 is persistently communicating with a single external IP address (185.244.25.235) over TCP port 6667. This port is standard for Internet Relay Chat (IRC), a protocol commonly used for botnet command and control (C&C). The logs are explicitly labeled as 'Malicious' and 'C&C', confirming the device is compromised and part of a botnet. The mix of established sessions (S2) and connection attempts (S0) is characteristic of C&C heartbeat and communication activity.

Fig. 5 Tabular View of Results

Furthermore, the reasoning provided by the LLM allows the security team to also audit the results of the LLM. A security team can review the reasoning, and cross reference the logs manually, or through a more autonomous method like a Splunk search. This additional context should effectively elicit additional confidence in the use of cloud-based LLMs as effective IoT triage agents.

4 - Discussion & Future Work

4.1 Operational Efficiency and Budget Trade-offs

The experimental findings detailed in this proof-of-concept demonstrate that the integration of cloud-based LLMs, like Gemini Pro, into security telemetry pipelines could drastically lower the manual triage time spent by security personnel. However, moving this framework from an academic proof-of-concept into a real-world production requires the navigation of immense fiscal constraints. While these models exhibit a high level of sophisticated context when analyzing log anomalies in IoT devices, operating Gemini Pro fully autonomously would scale data processing expenses by a factor of 16.6 compared to Gemini Flash. The long-term cost savings could be further reduced if organizations decide to opt for self-hosted language models. Deploying a locally hosted, open-source Small Language Model (SLM) could produce similar results, but would require the obtaining and managing of the underlying hardware and software. The initial purchasing of hardware to locally host an SLM could be a large financial undertaking which would be economically unviable for small-to-medium enterprises. Ultimately, this economic reality highlights the need for enterprise deployments to heavily regulate ingestion boundaries as architectural and financial constraints, rather than just parameters. For general triage, it may be more useful to utilize Gemini Flash rather than Gemini Pro, but the 1,000 line sampling is required to prevent immense financial costs. While the costs are far lower on Gemini Flash, for an enterprise level corporation these costs can still quickly balloon.

4.2 Architectural Vulnerabilities: LLM Hallucinations

A primary and largely grounded critique of moving to widespread adoption of LLMs within enterprise incident response would be the persistent risk of model hallucinations. A model hallucination is when the LLM provides information which simply does not exist. For example, in 2022 a customer of AirCanada was provided completely inaccurate information related to their refund policy. AirCanada stated that they were not liable to adhere to the policy, but Canadian federal courts ruled otherwise (Boykis, 2024). These hallucinations can cause real damage to enterprises utilizing LLMs for agentic workflows. A hallucination within the security space can cause false alerts, which may include IP addresses not even in the dataset to begin with. This could lead to all-hands-on-deck P0 responses, which are costly, for something that does not exist. In addition, these hallucinations could cause missed alerts. If the LLM falsely triages as low priority or non-threat, teams may not respond as quickly to legitimate threats.

To systematically mitigate this vulnerability, production-grade pipelines have a need to fully decouple open-ended generative reasoning from the log extraction. This design philosophy aligns with the research into lightweight, further specialized security models which incorporate localized knowledge retrieval to bound the probability of language model hallucinations (Hammar et al., 2025). Within the framework, the validation mechanism is built to limit overall context to the language model. This is done by forcing the underlying orchestration layer to process isolated, sequential strings via stream chunking libraries. This is to ensure the analytics engine is grounded within the underlying Zeek log attributes. Despite these efforts, it is still possible for hallucinations

to occur, which is a large long-term concern to the utilization of agentic LLMs in the cybersecurity space.

5 - Conclusion

This proof-of-concept was able to successfully design, implement, and evaluate the feasibility of utilizing cloud-based LLMs for anomaly detection and triage of heterogeneous IoT devices.

While the results are highly promising, there are still a number of concerns that must be addressed, especially if utilized at the enterprise level.

It cannot be understated that budget is a concern. The costs of cloud-based LLM tokens can quickly balloon to substantial numbers. With funding for SOC's already below what is necessary, it seems unlikely adoption for costly language models is on the horizon. Executive leadership often struggles to view the benefit of properly funding and staffing their security operation centers, as they struggle to see the return on investment (Cisometric, 2023). To reiterate, it is highly unlikely we see widespread adoption until the costs come down.

Despite the financial burden, the LLM still did quickly triage 1,000 lines quickly and efficiently. It is certainly a viable option, especially when provided with tight constraints to prevent the costs from ballooning to unexpected levels. As autonomous agents become increasingly central to cyber defense, the architectural principles established within this proof-of-concept provide a financially viable, scalable, and auditable roadmap to secure heterogeneous, distributed IoT ecosystems.

References

- Boykis, M. (2024, April 11). *LLM hallucinations and failures: Lessons from 5 examples*. Evidently AI. <https://www.evidentlyai.com/blog/llm-hallucination-examples>
- Cisometric. (2023). *Underinvestment in cybersecurity*. <https://www.cisometric.com/articles/underinvestment-in-cybersecurity>
- Garcia, S., Parmisano, A., & Erba, M. (2020). *IoT-23: A labeled dataset with malicious and benign IoT network traffic* (Version 1.0.0) [Data set]. Stratosphere Laboratory, Czech Technical University. <https://doi.org/10.5281/zenodo.4743778>
- Hammar, K., Alpcan, T., & Lupu, E. C. (2025). *Incident response planning using a lightweight large language model with reduced hallucination*. arXiv. <https://doi.org/10.48550/arxiv.2508.05188>
- Lacey, P. (2026, June 29). *The hidden cost of agentic AI: Why most projects still die before production*. Splunk Observability Blog. https://www.splunk.com/en_us/blog/observability/why-most-projects-still-die-before-production.html
- Lear, E., Droms, R., & Romascanu, D. (2019). *Manufacturer Usage Description (MUD) specification* (RFC Reference No. 8520). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc8520>
- Neto, E. C. P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., & Ghorbani, A. A. (2023). CICIOT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment. *Sensors*, 23(13), Article 5941. <https://doi.org/10.3390/s23135941>
- Searchlight Cyber. (2026, June 29). *Why preemptive cybersecurity matters*. Searchlight Cyber Blog. <https://slcyber.io/blog/why-preemptive-cybersecurity-matters/>
- Tejero-Fernández, R., & Sánchez-Macián, A. (2025). *Evaluating language models for threat detection in IoT security logs*. arXiv. <https://arxiv.org/abs/2507.02390>
- The MITRE Corporation. (2025). *MITRE ATT&CK for Enterprise* (Version 16). <https://attack.mitre.org/matrices/enterprise/>
- Verizon. (2025). *2025 Data Breach Investigations Report (DBIR)*. Verizon Business.

Google Gemini was also used to proof read each section, provide APA references, and assist with the coding of the [Node.js](#) app that was used to make the API call to Google Gemini